

E-commerce Product Recommendation System

Implementing recommender system, clustering and NLP

1st Devanshi Singhal
Dept. of Computer Applications
KIET Group of Institutions
Ghaziabad, India
devanshisinghal10@gmail.com

2nd Deepanshu Ruhela
Dept. of Computer Applications
KIET Group of Institutions
Ghaziabad, India
deepanshuruhela18@gmail.com

3rd Aanchal
Dept. of Computer Applications
KIET Group of Institutions
Ghaziabad, India
aanchalchaudhary435@gmail.com

4th Dhvani Panchal
Dept. of Computer Applications
KIET Group of Institutions
Ghaziabad, India
panchal.dhwani9760@gmail.com

5th Apoorv Jain
Assistant Professor, Dept. of Computer Applications
KIET Group of Institutions
Ghaziabad, India
apoorv.jain@kiet.edu

Abstract—E-commerce has transformed shopping by providing unmatched convenience and variety. However, the vast number of products makes it challenging for users to find relevant items, particularly without historical interaction data. This paper presents a content-based product recommendation system, “E-Commerce Product Recommendations using NLP and Clustering,” which employs Natural Language Processing (NLP) and clustering techniques for personalized suggestions. By extracting features from product metadata using TF-IDF vectorization, the system generates high-dimensional vectors that are compared through cosine similarity to identify semantically similar products. Clustering algorithms like K-Means further enhance performance by grouping related products for more efficient recommendations. A Flask-based web interface enables real-time interaction, and the architecture supports cold-start scenarios without relying on previous user behavior. This solution is ideal for small to mid-sized e-commerce platforms aiming to implement intelligent recommendation systems, with future enhancements potentially including hybrid models and real-time learning for improved adaptability.

Index Terms—Content-Based Recommendation, Natural Language Processing (NLP), Cosine Similarity, Clustering

I. INTRODUCTION

E-commerce platforms have significantly reshaped the retail landscape by providing consumers with extensive product selections and convenient access. However, users frequently encounter difficulties in identifying relevant items due to the overwhelming volume of choices and the limited degree of personalization, particularly in situations where prior interaction data is lacking. Traditional recommendation systems, which are predominantly based on historical user behavior, often prove inadequate in these “cold-start” scenarios. This paper proposes a novel content-based recommendation system that employs Natural Language Processing (NLP) techniques and clustering methods to suggest similar products based on their textual metadata. By vectorizing product descriptions utilizing the Term Frequency-Inverse Document Frequency (TF-IDF) and calculating cosine similarity, the proposed system

improves product discovery and generates personalized recommendations, all facilitated through an intuitive web interface.

II. PREVIOUS WORK

- Recommendation systems have evolved significantly, with collaborative filtering and content-based filtering being two of the most widely adopted approaches. Sarwar et al. [1] introduced item-based collaborative filtering, which emphasizes the history of user-item interactions. However, these methods often struggle in cold-start scenarios where user history is sparse or unavailable.
- To address these limitations, content-based approaches utilizing Natural Language Processing (NLP) have gained traction. Lops et al. [2] demonstrated how TF-IDF and semantic analysis can be used to extract meaningful features from product descriptions and reviews for the purpose of generating recommendations. More recent research has explored advanced models such as Word2Vec, BERT, and Doc2Vec, which capture deeper semantic relationships within textual data [3].
- Clustering techniques have also been integrated into recommendation engines to improve scalability and interpretability. For instance, Das et al. [4] applied K-means clustering to group similar products and users, facilitating faster and more relevant recommendations. The hybrid approach that combines clustering with NLP, as proposed by Aggarwal [5], enables product grouping based on semantic similarity rather than merely numerical features, thereby enhancing the quality of recommendations for both new and existing users. This project builds on these foundations by employing TF-IDF for feature extraction, cosine similarity for comparison, and K-means clustering for product grouping. This methodology allows for efficient, scalable, and context-aware product recommendations on an e-commerce platform.

III. OBJECTIVE

The objective of this project is to design and implement a content-based recommendation system for e-commerce platforms that leverages Natural Language Processing (NLP) and clustering techniques to enhance product discovery and personalization. The system aims to generate relevant product recommendations by analyzing textual metadata such as tags and descriptions, without relying on user interaction history. It addresses the cold-start problem by computing semantic similarities between products using TF-IDF vectorization and cosine similarity. Additionally, the system incorporates clustering to group similar products and improve the efficiency and contextual relevance of recommendations within a real-time web application.

IV. METHODOLOGY

The proposed system employs a content-based recommendation strategy that integrates Natural Language Processing (NLP) and clustering techniques to suggest relevant products based on their textual metadata. The methodology is divided into several stages as outlined below:

A. Data Collection and Preprocessing

The dataset comprises product information including titles, tags, and descriptions. Preprocessing is essential to ensure consistency and quality of input data. The following NLP techniques are applied:

- Tokenization
- Lowercasing
- Stopword removal
- Lemmatization

The cleaned text forms the basis for feature extraction. The raw dataset was loaded in tab-separated format using the Pandas library. Essential columns relevant to product information were selected, such as product ID, category, brand, rating, review count, and textual metadata (product name and description). Missing values in numeric fields like ratings and review counts were imputed with zero, while textual fields were filled with empty strings to ensure processing consistency. String-based IDs were cleaned using regular expressions and converted to numeric types for indexing. Redundant and duplicate records were removed, and column names were standardized for better readability and downstream compatibility.

B. Feature Extraction Using Clustering

After text preprocessing, each product description is transformed into a TF-IDF vector to numerically represent the importance of terms. These vectors are then input into a K-Means clustering algorithm to group semantically similar products into clusters. Each product is assigned a cluster label, which serves as a high-level semantic feature for recommendation generation. This clustering step reduces dimensionality and noise in the vector space by organizing products into coherent groups. The cluster assignments provide an additional feature that enables category-aware recommendations, improves scalability, and assists in delivering curated suggestions. To prepare

the product descriptions for content-based analysis, NLP techniques were applied using the spaCy library. Product metadata fields such as Category, Brand, and Description were cleaned by removing stop words, punctuation, and non-alphanumeric characters. Tokens were normalized to lowercase, and only semantically meaningful words were retained. These processed tokens were then aggregated into a new feature called Tags, representing a unified and enriched textual descriptor for each product. This Tags field was essential for computing textual similarity in the next stages.

C. Similarity Measurement

For a given product, the system computes cosine similarity between its TF-IDF vector and those of other products within the same cluster. This localized comparison ensures more contextually relevant recommendations by avoiding comparisons with unrelated items from distant clusters. Cosine similarity was used to compute the angle-based similarity between product vectors in the TF-IDF space. Products with the highest similarity scores are considered the most contextually relevant and are recommended to the user. A Python function encapsulated this logic, allowing dynamic recommendations based on real-time user input.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} \quad (1)$$

D. Web-Based Implementation

A Flask-based web application provides the front end for user interaction. Users can:

- Browse and search for products
- View similar product recommendations
- Register/login for personalized features
- The backend uses SQLite to manage user sessions and product data.

A web interface was built using the Flask framework to make the recommendation system interactive and user-friendly. Users can search for products, receive personalized recommendations, and interact with the application through intuitive pages such as a home page (showing top-rated items), a

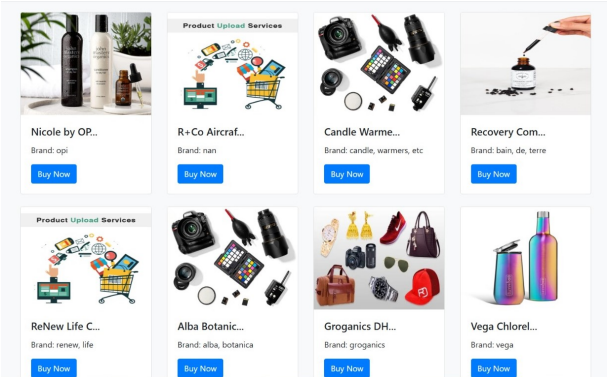


Fig. 1. Web Interface Displaying Personalized Product Recommendations
recommendation results page, and login/signup forms. The

backend uses SQLite with SQLAlchemy ORM to manage user sessions, product data, and recommendation results.

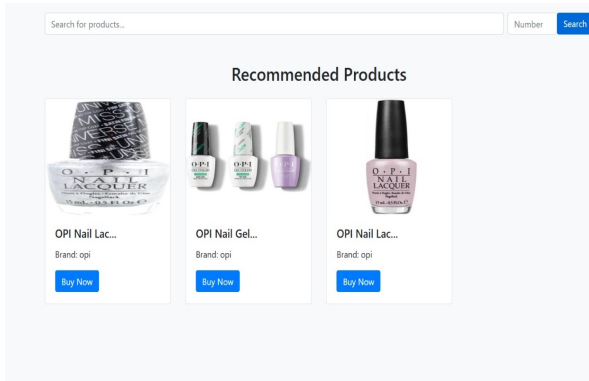


Fig. 2. Web Interface Displaying Recommended Product based on previous search.

E. Tools and Libraries

The implementation of the e-commerce recommendation system utilized a comprehensive set of open-source libraries and frameworks. These tools were selected for their efficiency, community support, and suitability for machine learning, natural language processing, and web development tasks. Below is a detailed overview of the key libraries and technologies employed in the project:

- **Python** Python served as the primary language due to its versatility, readability, and vast ecosystem of libraries. It provided the foundation for data handling, machine learning, NLP processing, visualization, and backend integration.
- **Pandas** Pandas was used extensively to load, clean, filter, and preprocess the dataset. Operations such as handling missing values, renaming columns, merging data, and generating statistical summaries were facilitated using Pandas DataFrames.
- **NumPy** NumPy provided support for efficient numerical operations and array-based computations, especially during matrix transformations and similarity calculations.
- **Scikit-learn** Scikit-learn played a central role in implementing:
 - TF-IDF Vectorization: To convert textual data into feature vectors.
 - Cosine Similarity: To measure similarity between product vectors and user preferences.
 - K-Means Clustering (for future scope): To group similar products based on their textual or numerical features.
 - Collaborative Filtering: To compute user similarity using the cosine similarity metric on the user-item matrix.
- **spaCy** The main purpose of using spaCy was to implement Natural Language Processing(NLP). SpaCy was used to process and clean product metadata such as descriptions, categories, and brand names. Tasks included:

- Tokenization
- Lowercasing
- Stopword removal
- Text normalization

This helped generate meaningful and noise-free tags for content-based filtering.

- **Matplotlib and Seaborn** These libraries were used to create:

- Histograms of user-item interactions
- Heatmaps to visualize the sparsity of the user-rating matrix
- Bar charts for most popular and highest-rated products

These visualizations were crucial for understanding dataset characteristics and guiding system design.

- **Flask** Flask was used to develop the web interface, enabling real-time interaction with users. It handled: route creation for home and recommendation pages, rendering of templates, and receiving and processing user input for personalized recommendations.

F. Recommendation Generation and workflow

The recommendation process begins when a user inputs a product name into the web interface. To handle misspellings or partial matches, the system employs fuzzy string matching to identify the closest valid product name from the database. Once matched, the corresponding product's tags are transformed into a vector representation using TF-IDF (Term Frequency–Inverse Document Frequency). The system then computes the cosine similarity between this vector and the vectors of all other products in the dataset. Based on the similarity scores, the top-N most similar products are selected. These recommendations are then dynamically rendered on the Flask-based user interface, providing users with personalized and contextually relevant suggestions in real time.

V. ANALYSIS AND RESULTS

A. Data Loading and Preprocessing

The initial phase of the project involved loading and preparing the dataset for further processing. The dataset, sourced in TSV (Tab-Separated Values) format, was imported using the Pandas library to create a structured DataFrame. To focus on relevant product-related attributes, only selected columns such as Uniq Id, Product Id, Product Rating, Product Reviews Count, Product Category, Product Brand, Product Name, Product Image Url, Product Description, and Product Tags were retained. This helped in reducing noise and ensured computational efficiency. Subsequently, missing values in critical fields were handled through imputation: numerical fields such as product rating and review count were filled with zero, while categorical and textual fields like product category, brand, and description were filled with empty strings to maintain data consistency. To enhance interpretability and ease of processing, verbose column names were renamed to concise forms—for example, Product Rating was renamed to

Rating and Product Brand to Brand. Additionally, string-based identifiers (Uniq Id and Product Id) were cleaned using regular expressions to extract numeric components, which were then converted to float data types. This standardized preprocessing pipeline ensured that the data was clean, consistent, and ready for downstream natural language processing and machine learning tasks in the recommendation system.

B. Exploratory Data Analysis(EDA)

Exploratory Data Analysis (EDA) was conducted to gain an initial understanding of the dataset’s structure, distribution, and potential anomalies. Basic statistical summaries were computed to determine the number of unique users, products , and rating values. This helped assess data sparsity, a common issue in recommendation systems where not all users rate all items. A pivot table was then constructed using user identifiers as rows and rating values as columns to visualize user-item interactions. This matrix formed the basis for generating a heatmap using *Seaborn*, which provided visual insights into rating density and user engagement patterns. The heatmap highlighted which rating values were most commonly given by users and helped identify segments with limited user activity. Histograms plotted the number of interactions per user and

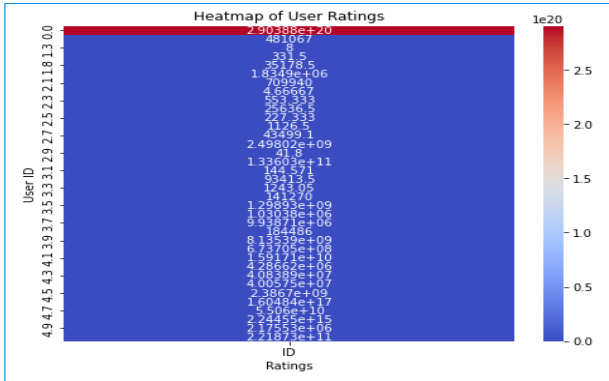


Fig. 3. Heatmap of User Ratings: This visualization represents the distribution of user ratings across different product IDs.

per item, revealing whether certain users or items dominated the dataset. This is crucial for understanding user behavior and item popularity. The results indicated that a small subset of products received the majority of ratings, which is typical in real-world e-commerce platforms. Additionally, the most popular products were identified based on the frequency of interactions, and the most frequently occurring rating values were visualized using bar plots. These insights guided the design of the recommendation logic by emphasizing popular products and user engagement trends, thereby aligning the system’s goals with actual user behavior patterns.

C. Tags and Rating-Based Recommendation

To enhance the effectiveness of content-based recommendations, the third step focused on cleaning and engineering meaningful product tags using Natural Language Processing (NLP) techniques. The product metadata—specifically the

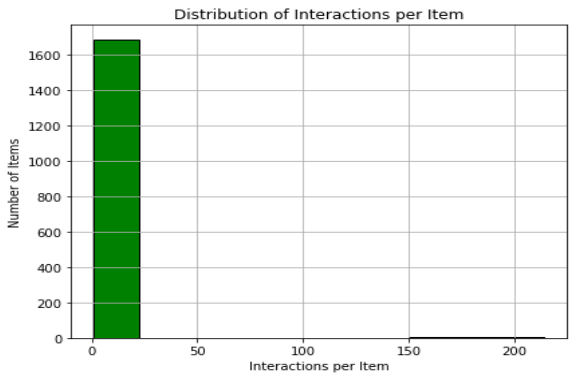


Fig. 4. User and Item Interaction Distributions: The subplot illustrates the distribution of interactions per user, indicating that most users interacted with only a few products.

Category, Brand, and Description fields—were preprocessed using the spaCy NLP library. This involved converting text to lowercase, removing stop words, and extracting only alphanumeric tokens to generate clean, informative keywords. These cleaned keywords were then concatenated to form a unified Tags field for each product, serving as the core input for the TF-IDF vectorization in later stages. This process enriched the textual representation of each product, enabling more accurate computation of semantic similarity.

Following this, a basic rating-based recommendation system was implemented represented Figure 5. The system computed

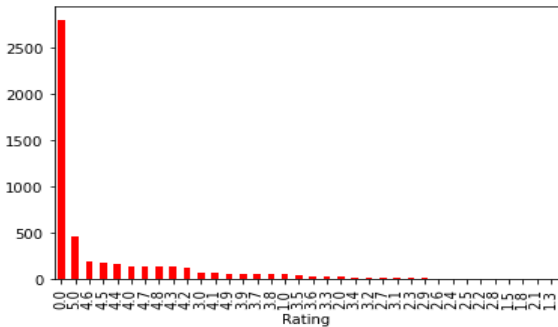


Fig. 5. Most Popular Products by Interaction Count: This bar chart displays the most frequently interacted-with products in the dataset.

the average rating for each product by grouping records based on Product Name, Review Count, Brand, and ImageURL. Products were then ranked in descending order of their average ratings to identify top-rated items. The top-N products were selected and presented as trending or highly rated recommendations. This approach provides a popularity-based baseline and serves as a useful supplement to the content-based filtering system, especially when item metadata is not sufficient for accurate recommendations. Together, these steps prepared the dataset for both semantic analysis and initial rating-based filtering, forming a hybrid recommendation foundation.

D. Content Based Recommendation and Function Development

In this step we focused on building and implementing a content-based recommendation system using Natural Language Processing (NLP) techniques. After cleaning and consolidating product metadata into a unified Tags field, the system utilized the TF-IDF (Term Frequency–Inverse Document Frequency) vectorization method to convert these tags into numerical feature vectors. This transformation captured the semantic importance of each term within the context of the product corpus. Once vectorized, cosine similarity was applied to compute the pairwise similarity between all product vectors. This allowed the system to identify items with similar textual descriptions based on the angular distance between their vector representations. To operationalize the recommendation engine,

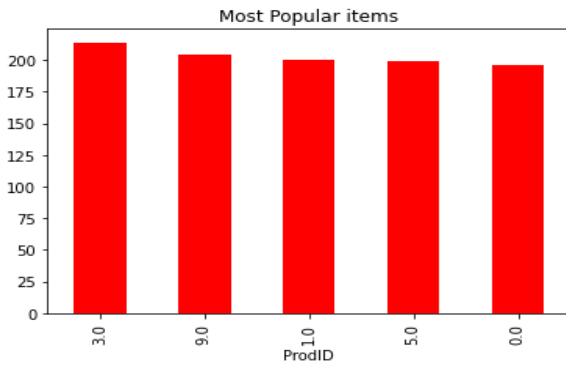


Fig. 6. Top-Rated Products Based on Average User Ratings: This bar chart displays the products with the highest average ratings in the dataset.

a reusable function named 'content based recommendations()' was developed. This function accepts a product name as input, checks its presence in the dataset, and returns the top-N most contextually similar products. It performs TF-IDF vectorization on all product tags, locates the input item's vector, and computes cosine similarity scores with every other product. It then sorts the results and retrieves the highest-ranking similar items, excluding the item itself. This modular function enables scalable and real-time generation of recommendations based solely on item content, making it especially effective in cold-start scenarios where user interaction data is limited or unavailable.

VI. CONCLUSION

This project successfully developed a scalable and intelligent content-based recommendation system for e-commerce platforms by integrating Natural Language Processing (NLP), cosine similarity, and clustering techniques. The system efficiently addresses the cold-start problem by relying on product metadata—such as tags, descriptions, categories, and brand information—rather than user behavior or historical interaction data. Through the use of TF-IDF vectorization and semantic similarity measures, the system is capable of suggesting contextually relevant products in real time. Exploratory

Data Analysis and visualization techniques provided insights into user interaction patterns, product popularity, and rating distributions, which further informed system design. The implementation of content-based and collaborative filtering modules, along with a unified hybrid approach, demonstrated the flexibility and effectiveness of the recommendation engine. Additionally, a web-based interface using Flask and SQLite ensured usability, interactivity, and deployment readiness. The modular architecture and use of open-source tools enable easy extension of the system to incorporate advanced models such as deep learning or real-time collaborative learning in future work. Ultimately, this project lays a solid foundation for enhancing user experience and product discoverability in modern e-commerce platforms.

REFERENCES

- [1] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," *Proceedings of the 10th International Conference on World Wide Web*, pp. 285–295, 2001.
- [2] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, Springer, pp. 73–105, 2011.
- [3] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [4] A. S. Das, M. Datar, A. Garg, and S. Rajaram, "Google news personalization: scalable online collaborative filtering," *Proceedings of the 16th international conference on World Wide Web*, pp. 271–280, 2007.
- [5] C. C. Aggarwal, *Recommender Systems: The Textbook*, Springer, 2016.
- [6] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967. [K-Means Clustering]
- [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," *To appear*, 2017. [spaCy Library]
- [9] Python Software Foundation, "Python Language Reference," [Online]. Available: <https://www.python.org>
- [10] Flask Documentation, "Flask Web Framework," [Online]. Available: <https://flask.palletsprojects.com/>
- [11] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, 1980. [Porter Stemmer used in NLP]
- [12] SQLite Consortium, "SQLite Documentation," [Online]. Available: <https://www.sqlite.org>
- [13] W. Cohen, "Fast effective rule induction," in *Machine Learning Proceedings 1995*, Elsevier, pp. 115–123. [On rule-based data processing, relevant to fuzzy matching]